GUÍA DE TRABAJO PARA EL ESTUDIANTE

1. EL RETO DEL ARQUITECTO DE SOFTWARE

¡Estimados ingenieros en formación! Hasta el momento hemos estudiado cómo el software nace de la necesidad de modelar la realidad. Hoy, la cooperativa agroecológica de la "Comercializadora Amazónica" les entrega la versión 1.0 de su sistema de facturación e inventario.

El código es funcional y compila en NetBeans, pero un buen ingeniero de sistemas sabe que el software nunca está "terminado". Su misión para esta unidad es asumir el rol de **Arquitectos de Mantenimiento y Evolución**. Deberán ejecutar el sistema, identificar cuellos de botella operativos, proponer mejoras (técnicas, visuales o de reglas de negocio) y programarlas directamente sobre el código fuente en Java, garantizando que el sistema sea más robusto para nuestros productores locales.

2. RUTA DE TRABAJO (FASES DEL RETO)

FASE 1: Auditoría y Empatía (Comprendiendo el Sistema)

- **Ejecución Base:** Descarguen el proyecto base, ábranlo en su IDE (NetBeans) y ejecuten el simulador.
- **Análisis del "Mundo del Problema":** Naveguen por el código y comprendan cómo colaboran las clases Factura, Producto y Cliente.
- **Identificación de Mejoras (Design Thinking):** Pónganse en los zapatos del usuario final (el cajero o el administrador de la tienda amazónica) y respondan: *¿Qué le falta a este sistema?*. ¿Falta aplicar un descuento especial a estudiantes de la Uniamazonia? ¿Se requiere una interfaz gráfica (GUI)? ¿Faltan persistencias en bases de datos o alertas de inventario crítico?

FASE 2: Consultoría Tecnológica (Uso Estratégico de Inteligencia Artificial)

Para este proyecto **tienen total libertad de utilizar asistentes de Inteligencia Artificial** (ChatGPT, Gemini, Claude, NotebookLM, etc.) para acelerar su desarrollo. Sin embargo, deben acatar esta regla inquebrantable de la ingeniería:

- **La IA es su consultora, ustedes son los arquitectos.**
- No copien ciegamente las respuestas de la IA. Evalúen en equipo si los atributos, métodos o bloques de código propuestos por la IA respetan las reglas de la Programación Orientada a Objetos (POO) vistas en clase.
- Utilicen la IA para: Explicar errores de compilación, refactorizar métodos largos, generar ideas de interfaz gráfica o aprender a implementar nuevas librerías.

FASE 3: Codificación y Refactorización (Manos a la Obra)

- Implementen sus mejoras en el paquete del mundo o de interfaz.
- **Rigor Técnico:** Toda nueva línea de código debe respetar el encapsulamiento estricto (private para atributos, public para métodos) y las convenciones de escritura (UpperCamelCase para clases, lowerCamelCase para variables y métodos).
- Asegúrense de realizar las validaciones necesarias para evitar referencias inexistentes (como el temido NullPointerException) y operaciones inválidas.

FASE 4: Documentación y Sustentación

- El código no se explica solo. Siguiendo los estándares de calidad de la ingeniería de software, toda modificación debe estar documentada para garantizar que futuros desarrolladores puedan aprender a utilizarlo y mantenerlo.
- Preparen una **Bitácora de Cambios (Change Log)** donde detallen: *¿Qué mejoraron? ¿Por qué lo mejoraron? ¿Cómo utilizaron la IA para lograrlo?*

3. EVIDENCIAS ESPERADAS Y ENTREGABLES

1. El proyecto en NetBeans ejecutable, limpio y comentado.
2. Un informe corto justificando las funcionalidades agregadas o refactorizadas y cómo impactan positivamente en el contexto de la Comercializadora Amazónica.
3. Deben sustentar por qué tomaron esas decisiones de diseño y explicar línea a línea cómo el código nuevo se integra con el modelo original. ¡El código debe ser consecuencia de su modelo lógico!